

Ejercicio propuesto de Bash. Curso 26-27.

Objetivos

Es hora de aprender a manejar los conceptos vistos en semanas anteriores para crear scripts por nosotros mismos que automaticen las tareas más comunes.

Evaluación

La evaluación de este trabajo se realizará con una prueba presencial práctica de tipo individual, donde tendrás que resolver problemas similares a los presentados aquí. En esa prueba **no dispondrás del código desarrollado para resolver los ejercicios que aquí se proponen**. Se realizará en un ordenador sin conexión a internet y dispondrás de los apuntes sobre bash del aula virtual, así como de las páginas de la herramienta man.

Introducción: comando lsof

El comando **lsof** se utiliza para listar todos los archivos abiertos en el sistema. El comando es complejo y admite diversas opciones. Una de las más interesantes es **-u**, que permite conocer todos los archivos abiertos por un usuario concreto:

```
$ lsof -u root
```

La salida tiene una línea por cada archivo abierto, con estas columnas:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	2114	dabreu	cwd	DIR	254,0	4096	950300	/home/dabreu
bash	2114	dabreu	txt	REG	254,0	1446024	343921	/usr/bin/bash
bash	2114	dabreu	mem	REG	254,0	2129424	344580	/usr/lib/x86...
bash	2114	dabreu	0u	CHR	136,1	0t0	4	/dev/pts/1

Para esta práctica interesan tres: USER (el propietario del proceso), TYPE (el tipo de archivo) y NAME (la ruta del archivo, que siempre es la última columna).

Ten en cuenta dos comportamientos de lsof:

1. Sin privilegios de administrador, lsof solo puede ver los archivos abiertos por tus propios procesos. Para los demás usuarios, la cuenta será 0. Es el comportamiento esperado y no un fallo de tu script.
2. lsof termina con código de salida 1 cuando ningún archivo cumple los criterios indicados. Eso no es un error. Significa que no han encontrado ninguno.

Tareas

El script a realizar se llamará `open_files`, y admite las siguientes opciones:

`open_files [-h | --help] [-f regexp] [-o] [--offline] [-u usuario ...] [-s ] [--sort]`

En la notación anterior, las letras precedidas por un guión como `-h`, o de dos guiones como `--offline` son opciones. Las que van después de un guión se llaman opciones en formato corto (una letra), mientras que las que van después de dos guiones se llaman opciones en formato largo. A veces una misma funcionalidad está expresada por la opción en formato corto y también en formato largo, por ejemplo `-h` y `--help`, ya que ambas indican que se desea mostrar la ayuda del comando. En la notación, la barra vertical separa las dos formas de una misma opción y los corchetes indican que es opcional.

Observa que las opciones pueden combinarse e introducirse en cualquier orden.

Por ejemplo,

```
$ open_files -s -u dabreu jttoledo -f '\.log$'
```

y

```
$ open_files -f '\.log$' --sort -u dabreu jttoledo
```

deben dar el mismo resultado.

El script implementará las siguientes funcionalidades:

1. Funcionamiento básico y listado de usuarios con archivos abiertos

Usando lo aprendido hasta el momento en clase, **desarrolla el script 'open_files' que, si no se especifica opción, envía a la salida estándar una lista ordenada alfabéticamente por el nombre de usuario y sin duplicados de todos los usuarios conectados en el sistema y la cuenta del número de ficheros regulares abiertos que tiene actualmente cada usuario.** Los archivos regulares se especifican por `lsuf` con las letras `REG` en la columna `TYPE`. Se cuenta cada línea de `lsuf` de tipo `REG`, aunque la misma ruta aparezca varias veces (por ejemplo, una biblioteca cargada por varios procesos).

En este comportamiento por defecto, este listado estará formado por los usuarios que se incluyen en el resultado del comando `who`, y no usuarios como demonios o utilizados por servicios que no han hecho login. También se debe incluir el UID del usuario, y el PID de su proceso más antiguo. El proceso más antiguo es, de todos los procesos del usuario, el que lleva más tiempo en ejecución, tenga o no archivos abiertos. Consulta las páginas de manual de `ps` y `pgrep` para ver cómo obtenerlo. Recuerda que `who` muestra una línea por sesión, así que un mismo usuario puede aparecer varias veces.

La salida será una tabla con una línea de cabecera y una línea por usuario, con cuatro columnas separadas por espacios y en este orden: nombre de usuario, UID, PID del proceso más antiguo y número de archivos regulares abiertos. Por ejemplo, ejecutado con privilegios de administrador:

```
$ ./open_files
USUARIO      UID  PID_ANTIGUO  N_ARCHIVOS
dabreu       1002    2114        27
jmtorres     1000    1830       143
```

No hace falta que las columnas queden alineadas exactamente como en el ejemplo, pero sí que aparezcan en ese orden. El comando `printf` te ayudará a alinearlas.

El script debe empezar por la línea `#!/bin/bash` (o similar) y tener permiso de ejecución.

Requisitos generales

Los siguientes requisitos se aplican a todo el script, no solo a este apartado:

1. **Recuerda facilitar la lectura de tu código:**
 1. Usa comentarios en los elementos más complejos del programa para que te sea más sencillo entenderlo cuando haya pasado un tiempo y no te acuerdes
 2. Usa funciones para compartimentar el programa, así como variables y constantes (variables en mayúsculas) que hagan tu código más legible.
2. **Incluye código para procesar la línea de comandos.**
 1. Se debe mostrar ayuda sobre el uso del comando si el usuario emplea la opción `-h` o `--help`. En ese caso el script termina con código de salida 0 sin hacer nada más.
 2. Se debe indicar el error y mostrar ayuda sobre el uso si el usuario emplea una opción no soportada. Los mensajes de error se envían a la salida de error estándar.
3. **Ojo con la sustitución de variables y las comillas.** En caso de problemas, piensa en cómo quedarían las sentencias si las variables no valieran nada, ¿tendría sentido para BASH el comando a ejecutar?
4. **Maneja adecuadamente los errores.**
 1. En caso de error muestra un mensaje y sal con código de salida distinto de 0. Recuerda la función **`error_exit`** explicada en los apuntes y, si lo crees conveniente, reutilízala o haz la tuya propia.
 2. Trata como un error que el usuario emplee opciones no soportadas, argumentos sueltos que no pertenezcan a ninguna opción (`open_files foo`) u opciones a las que les falte su argumento (`-f` sin expresión regular, `-u` sin ningún usuario).
 3. Detecta si **`lsuf`** está instalado antes de usarlo. Si no lo está, indica el error.
 4. Haz lo mismo con las otras posibles condiciones de error que se te ocurran ¿Has probado a invocar tu programa con opciones absurdas a ver si lo haces fallar?
 5. Que un usuario no tenga archivos abiertos no es un error: aparece en la tabla con cuenta 0.

2. Variable de entorno OPEN_FILES_FOLDER

En el caso de que la variable de entorno OPEN_FILES_FOLDER esté asignada a la ruta de un directorio, se entenderá que la cuenta de archivos solo incluirá aquellos archivos dentro de ese directorio o de cualquiera de sus subdirectorios.

Comprueba primero si la variable no es nula y en el caso de que no lo sea, que la ruta lleve a un directorio existente. Si la variable no es nula, pero la ruta no existe o no es un directorio, hay que terminar el script con un estado de error.

Para que el script vea la variable, esta tiene que estar en su entorno. Puedes exportarla antes de llamarlo:

```
$ export OPEN_FILES_FOLDER=/var/log
$ ./open_files
```

O asignarla en la misma línea de la llamada:

```
$ OPEN_FILES_FOLDER=/var/log ./open_files
```

El comando **ls** puede combinar los criterios de elección de archivos abiertos, por ejemplo los pertenecientes a un usuario o los existentes en un directorio. Por defecto utiliza un **o** lógico, pero en este caso puede que te interese más un **and** lógico, en cuyo caso en la línea de comando de **ls** debes añadir la opción **-a**. Para seleccionar los archivos abiertos dentro de un directorio, mira las opciones **+d** y **+D** en la página de manual: pasar a **ls** la ruta del directorio sin más no lista su contenido. Al recorrer el directorio, **ls** puede escribir avisos en la salida de error estándar, que no deben aparecer en la salida de tu script.

3. Busca archivos según un patrón

La última columna de **ls** muestra la ruta del archivo abierto.

Añade la opción '-f filtro' de tal forma que filtro pueda ser una expresión regular. Esta expresión se usaría para filtrar la salida de ls en base a la última columna. Es decir que al usar tu script así:

```
open_files -f '\.sh$'
```

En lugar de contar todos los ficheros abiertos, se contarán solo aquellos en los que la ruta encaje con esta expresión regular. En este ejemplo, los archivos cuya ruta termina en **.sh**.

El filtro es una expresión regular extendida (la sintaxis de **grep -E**) y se compara solo con la ruta, no con la línea completa de **ls**: un filtro como **bash** no debe contar un archivo por el hecho de que el comando que lo abrió se llame **bash**. Basta con que la expresión encaje con una parte de la ruta. Si quiere exigir que encaje al principio o al final, el usuario puede usar **^** y **\$**. Puedes suponer que las rutas no contienen espacios.

4. Ficheros de Usuarios no conectados

Si se incluye el parámetro `-o` o `--offline` en su versión larga se debe mostrar el listado anterior, pero únicamente para los usuarios que no estén conectados al sistema, es decir, **aquellos usuarios definidos en `/etc/passwd` que no salgan en el resultado de `who`. Si el usuario no tiene ningún proceso en ejecución, se mostrará la cadena `NA` en el campo del `PID`.**

En este modo sí aparecerán los usuarios del sistema y de los servicios (`daemon`, `www-data`, `nobody`...), muchos de ellos sin procesos ni archivos abiertos:

```
$ ./open_files -o
USUARIO      UID  PID_ANTIGUO  N_ARCHIVOS
bin           2      NA           0
daemon       1      NA           0
jtttoledo    1001    2051         23
root          0        1          130
www-data     33     812          12
...
```

5. Filtra por usuario

Si el script se invoca así:

```
open_files -u jmtorres jtttoledo dabreu
```

donde la opción `-u` va acompañada de una lista de nombres de usuario de tamaño arbitrario y separados por espacios, solo se mostrarán en la tabla los usuarios especificados en la opción `-u`. La lista termina en el siguiente argumento que empiece por un guion, o al acabarse los argumentos. Si alguno de los usuarios indicados no existe en el sistema, el script termina con un error. Esta opción debe poderse combinar con el resto de opciones, incluida la opción `-f`. Esto implica que si se utiliza sin la opción `-o` solo se mostrarán en la tabla los usuarios conectados (incluidos por `who`) mientras que con la opción `-o` solo se mostrarán en la tabla los usuarios no conectados (los que no se obtengan con `who`)

Por ejemplo, si `jmtorres` y `dabreu` están conectados y `jtttoledo` no:

```
$ ./open_files -u jmtorres jtttoledo dabreu
USUARIO      UID  PID_ANTIGUO  N_ARCHIVOS
dabreu       1002    2114         27
jmtorres     1000    1830        143
```

```
$ ./open_files -o -u jmtorres jtttoledo dabreu
USUARIO      UID  PID_ANTIGUO  N_ARCHIVOS
jtttoledo    1001    2051         23
```

6. Ordenamiento

Si se utiliza la opción `-s` o `--sort`, la lista de usuarios debe mostrarse ordenada por el número de archivos abiertos, de mayor a menor. Los usuarios con el mismo número de archivos se ordenan alfabéticamente entre sí. La cabecera sigue siendo la primera línea.

Comprueba tu script

Antes de dar el script por terminado, prueba al menos estos casos:

LLAMADA	RESULTADO ESPERADO
<code>./open_files -h</code>	Muestra la ayuda. Código de salida 0.
<code>./open_files -x</code>	Error y ayuda por la salida de error. Código distinto de 0.
<code>./open_files foo</code>	Error: argumento no reconocido.
<code>./open_files -f</code>	Error: falta la expresión regular.
<code>./open_files -u</code>	Error: falta la lista de usuarios.
<code>./open_files -u noexiste</code>	Error: el usuario no existe.
<code>OPEN_FILES_FOLDER=/noexiste ./open_files</code>	Error: el directorio no existe.
<code>OPEN_FILES_FOLDER=/etc/passwd ./open_files</code>	Error: la ruta no es un directorio.
<code>./open_files -s -o -f '\.so' -u root nobody</code>	Tabla correcta con todas las opciones combinadas.
<code>./open_files; echo \$?</code>	0, aunque algún usuario tenga 0 archivos.

Comandos que te pueden resultar útiles: `who`, `id`, `ps`, `pgrep`, `grep`, `awk`, `cut`, `sort`, `uniq`, `wc`, `comm` y `printf`. Todos tienen página de manual.